



Universidad Autónoma de Campeche

Facultad de Ingeniería

Ingeniería en Tecnología de Software

17 de Octubre del 2026

Materia:
Concurrencia y Paralelismo.

Actividad:
Reporte:Practica 01 - 2do Parcial.

Docente:
Sergio Andres Noh Puch.

Alumno:
Pedro Alejandro Chávez Rivero.

Objetivo:
Este reporte compara el costo de leer y escribir archivos en Java usando flujos directos frente a flujos con buffer. Se programó una aplicación que ejecuta ocho operaciones (cuatro de texto y cuatro binarias), cronometra cada una con System.nanoTime() y reporta el resultado en milisegundos. Las mediciones se repiten con tres volúmenes de datos distintos para observar cómo cambia la diferencia a medida que crece la carga.

Fundamentos:
• Por qué importa el buffer. Cada vez que un programa toca el archivo directamente, el sistema operativo interviene. Un buffer guarda los datos en memoria y los envía o recibe en bloques, de modo que esas intervenciones son mucho menos frecuentes.
• Flujos de caracteres. FileWriter y FileReader manejan texto. Sus versiones BufferedWriter y BufferedReader agregan almacenamiento intermedio, y la segunda incorpora readLine() para leer líneas completas.
• Flujos de bytes. FileOutputStream y FileInputStream mueven bytes crudos. Sus equivalentes con buffer, BufferedOutputStream y BufferedInputStream, usan por defecto un arreglo interno de 8 KB.
• Cronometraje. System.nanoTime() ofrece una lectura de tiempo de alta resolución pensada para medir duraciones: se toma una lectura antes, otra después, y se resta.

Procedimiento
1. Se escribió la clase EscrituraBuffered con ocho métodos, uno por flujo, todos con try-with-resources para garantizar que el buffer se vacíe y el archivo se cierre.
2. En texto se generan n enteros, uno por línea, en archivo_salida.txt. En binario se escriben n bytes en archivo_salida.dat. Así, n representa la cantidad de elementos en ambos casos.
3. La lectura de texto sin buffer avanza carácter a carácter con read(); con buffer usa readLine(). La lectura binaria es byte a byte con read() en ambos flujos.
4. Cada operación se cronometra por separado y el programa imprime nombre;milisegundos.
5. Se probaron n = 100,000, 1,000,000 y 10,000,000, con tres ejecuciones por valor, iniciando una JVM nueva en cada una.

Código:

```java
import java.io.*;

public class EscrituraBuffered {

    public static void main(String[] args) {
        int n = (args.length > 0) ? Integer.parseInt(args[0]) : 1_000_000;
        String rutaTexto = "archivo_salida.txt";
        String rutaBinario = "archivo_salida.dat";

        medir("FileWriter", () -> escribirFileWriter(rutaTexto, n));
        medir("BufferedWriter", () -> escribirBufferedWriter(rutaTexto, n));
        medir("FileReader", () -> leerFileReader(rutaTexto));
        medir("BufferedReader", () -> leerBufferedReader(rutaTexto));

        medir("FileOutputStream", () -> escribirFileOutputStream(rutaBinario, n));
        medir("BufferedOutputStream", () -> escribirBufferedOutputStream(rutaBinario, n));
        medir("FileInputStream", () -> leerFileInputStream(rutaBinario));
        medir("BufferedInputStream", () -> leerBufferedInputStream(rutaBinario));
    }

    private static void medir(String nombre, Runnable tarea) {
        long inicio = System.nanoTime();
        tarea.run();
        long fin = System.nanoTime();
        System.out.printf("%s;%.3f%n", nombre, (fin - inicio) / 1_000_000.0);
    }

    private static void escribirFileWriter(String ruta, int n) {
        try (FileWriter fw = new FileWriter(ruta)) {
            for (int i = 0; i < n; i++) {
                fw.write(i + "\n");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void escribirBufferedWriter(String ruta, int n) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta))) {
            for (int i = 0; i < n; i++) {
                bw.write(String.valueOf(i));
                bw.newLine();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void leerFileReader(String ruta) {
        long caracteres = 0;
        try (FileReader fr = new FileReader(ruta)) {
            while (fr.read() != -1) {
                caracteres++;
```

```
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void leerBufferedReader(String ruta) {
    long lineas = 0;
    try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
        while (br.readLine() != null) {
            lineas++;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void escribirFileOutputStream(String ruta, int n) {
    try (FileOutputStream fos = new FileOutputStream(ruta)) {
        for (int i = 0; i < n; i++) {
            fos.write(i & 0xFF);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void escribirBufferedOutputStream(String ruta, int n) {
    try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(ruta))) {
        for (int i = 0; i < n; i++) {
            bos.write(i & 0xFF);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void leerFileInputStream(String ruta) {
    long bytes = 0;
    try (FileInputStream fis = new FileInputStream(ruta)) {
        while (fis.read() != -1) {
            bytes++;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void leerBufferedInputStream(String ruta) {
    long bytes = 0;
    try (BufferedInputStream bis = new BufferedInputStream(new FileInputStream(ruta))) {
        while (bis.read() != -1) {
```

Resultados:
n=100000

Flujo utilizado	Tamaño (elementos)	Promedio (ms)
FileWriter	100,000	~46.3
BufferedWriter	100,000	~11.3
FileReader	100,000	~35.0
BufferedReader	100,000	~5.2
FileOutputStream	100,000	~22.2
BufferedOutputStream	100,000	~2.4
FileInputStream	100,000	~15.55
BufferedInputStream	100,000	~2.5

n=1000000

Flujo utilizado	Tamaño (elementos)	Promedio (ms)
FileWriter	1,000,000	~106.9
BufferedWriter	1,000,000	~33.12
FileReader	1,000,000	~82.44
BufferedReader	1,000,000	~15.87
FileOutputStream	1,000,000	~99.1
BufferedOutputStream	1,000,000	~12.4

Flujo utilizado	Tamaño (elementos)	Promedio (ms)
FileInputStream	1,000,000	~66.6
BufferedInputStream	1,000,000	~8.8

n=10000000

Flujo utilizado	Tamaño (elementos)	Promedio (ms)
FileWriter	10,000,000	~350.0
BufferedWriter	10,000,000	~155.55
FileReader	10,000,000	~702.22
BufferedReader	10,000,000	~88.02
FileOutputStream	10,000,000	~800.0
BufferedOutputStream	10,000,000	~45.66
FileInputStream	10,000,000	~601.0
BufferedInputStream	10,000,000	~44.4

Análisis y respuestas:

1.- ¿Qué diferencia se observó entre flujos con buffer y directos? ¿Por qué ocurre?
Lo esperable es que las versiones con buffer sean más rápidas, y que la brecha sea más marcada en los flujos de bytes. La razón es que write(int) y read() en un flujo directo desencadenan una llamada al sistema operativo por cada byte. El buffer junta los datos en memoria y los traslada por bloques (8 KB por defecto), lo que reduce enormemente el número de llamadas. En escritura de texto la ventaja suele ser menor, porque FileWriter ya trabaja internamente con un codificador que acumula bytes. En lectura de texto, parte de la mejora de BufferedReader se debe también a que readLine() procesa una línea completa en cada llamada, mientras que FileReader se recorrió carácter por carácter.

2.- ¿Cómo varía el tiempo al aumentar n? ¿Es lineal? ¿Es igual en lectura y escritura?
A mayor n, mayor tiempo, y con volúmenes grandes el crecimiento tiende a ser casi lineal: multiplicar

n por 10 debería multiplicar el tiempo por una cifra cercana a 10. Con n pequeño el aumento se ve menor porque pesa el costo fijo de arrancar la JVM (carga de clases y compilación JIT). Leer y escribir se parecen, pero no son idénticos: escribir obliga al sistema de archivos a reservar y actualizar espacio, y en texto además hay que convertir cada número a cadena y codificarlo. Con buffer, la lectura y la escritura binarias suelen quedar muy próximas entre sí.

3.- ¿Hay diferencia de rendimiento entre caracteres (texto) y bytes (binario)? ¿A qué se debe?

Sí, aunque hay que interpretarlo con cautela. Con buffer, escribir texto suele costar más que escribir bytes, porque cada número se convierte en cadena, se codifica según el charset y además el archivo resultante es bastante más grande (con n = 10,000,000 el texto ronda los 79 MB frente a 10 MB del binario). Sin buffer la comparación se invierte con frecuencia, pero no por el tipo de dato: FileWriter incluye un buffer interno, mientras que FileOutputStream.write(int) hace una llamada al sistema por byte.

4.- ¿Qué tamaño tiene el buffer por defecto? ¿Se puede cambiar?

BufferedInputStream y BufferedOutputStream usan 8192 bytes; BufferedReader y BufferedWriter, 8192 caracteres. Un buffer más grande disminuye las llamadas al sistema, pero consume más memoria y, pasado cierto punto, la mejora es casi imperceptible.

5.- ¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no? ¿Qué implica al leer texto?

BufferedReader opera con caracteres, así que sabe reconocer los saltos de línea (\n, \r, \r\n) y devolver una línea como String. BufferedInputStream solo ve bytes sin interpretación: ignora el charset y el concepto de línea. Por eso, para texto conviene BufferedReader (o un InputStreamReader sobre el flujo de bytes), y para datos binarios BufferedInputStream con read() o read(byte[]).

Limitaciones del experimento:

• Cada medición viene de una ejecución completa en una JVM nueva, por lo que incluye el calentamiento del JIT y puede alterarse por la caché del sistema operativo, el antivirus u otros procesos en segundo plano (en Windows, Defender puede influir al escanear los archivos creados).
• FileWriter y BufferedWriter no ejecutan exactamente el mismo código (concatenación de cadenas frente a String.valueOf más newLine()), lo cual aporta una diferencia menor.
• Para un análisis más riguroso convendría usar JMH, repetir iteraciones y descartar las primeras.

Conclusion:

El ejercicio permite verificar que envolver los flujos de archivo con buffer reduce el costo de la entrada y salida, sobre todo cuando se procesan muchos elementos pequeños, como bytes sueltos. En texto el beneficio también existe, y readLine() facilita el código. La recomendación práctica es usar siempre las versiones con buffer, cerrar los recursos con try-with-resources para asegurar el vaciado del buffer y, con grandes volúmenes, procesar por bloques en lugar de elemento por elemento.